



STEAM



SteamVision



AI-Powered Analytics and Prediction Engine

<< Capstone Project >>



Task 1 : Web Scraping Steam Store

```
!pip install requests
!pip install beautifulsoup4
!pip install lxml
!pip install selenium
!pip install webdriver-manager
```

< Installing all the necessary libraries required for Web Scraping Steam. Individually they work as follows :

requests → Sends HTTP requests and downloads webpage content.

beautifulsoup4 → Parses HTML/XML and extracts required data from webpages.

lxml → Fast parser used with BeautifulSoup for efficient HTML/XML processing.

selenium → Automates a web browser for interacting with dynamic JavaScript-based websites.

webdriver-manager → Automatically installs and manages browser drivers needed by Selenium.

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
```



Task 1 : Web Scraping Steam Store

The last code in the previous page imports all the required libraries including :

time → Adds delays or manages timing during scraping.

webdriver → Launches and controls a browser using Selenium.

By → Helps locate webpage elements (buttons, text, links, etc.).

Service → Sets up and manages the Chrome browser driver service.

ChromeDriverManager → Automatically downloads and manages the ChromeDriver needed for Selenium.

```
url = https://store.steampowered.com/search/
headers = {"User-Agent": "Mozilla/5.0"}
response = requests.get(url, headers=headers)
print(response.status_code)
```

200

< The code on the left stores the Steam search page URL, sends a request to the website using a browser-like User-Agent, retrieves the webpage data, and prints the HTTP status code to check whether the connection was successful (e.g., 200 → Success, 404 → Page not found, 403 → Access denied, etc.). The output is given below the code.

```
# Launching Chrome.
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
# Opening Steam.
driver.get(url)
# Adding a delay for the page loading.
time.sleep(5)
```



Task 1 : Web Scraping Steam Store

The last code in the previous page launches the Chrome browser using Selenium, automatically installs the required ChromeDriver, opens the Steam webpage, and waits for 5 seconds so the page can fully load before further actions are performed.

```
# Scrolling multiple times.
for i in range(19):
    driver.execute_script("window.scrollTo(0,
document.body.scrollHeight);") # Scrolling to the bottom.
    print(f"Scrolled {i+1} time(s)")
    time.sleep(3) # Waiting for new games to load.
```

^ The code above repeatedly scrolls to the bottom of the Steam webpage 19 times using Selenium, prints the number of times scrolled, and waits 3 seconds after each scroll so new game content can load dynamically. The output is given right beside.

```
html = driver.page_source
soup = BeautifulSoup(html, "lxml")
soup
```

Scrolled 1 time(s)
Scrolled 2 time(s)
Scrolled 3 time(s)
Scrolled 4 time(s)
Scrolled 5 time(s)
Scrolled 6 time(s)
Scrolled 7 time(s)
Scrolled 8 time(s)
Scrolled 9 time(s)
Scrolled 10 time(s)
Scrolled 11 time(s)
Scrolled 12 time(s)
Scrolled 13 time(s)
Scrolled 14 time(s)
Scrolled 15 time(s)
Scrolled 16 time(s)
Scrolled 17 time(s)
Scrolled 18 time(s)
Scrolled 19 time(s)



Task 1 : Web Scraping Steam Store

The last code in the previous page gets the current webpage HTML from the browser, uses BeautifulSoup with the lxml parser to parse the HTML content, and stores it in soup so the webpage data can be easily extracted and analyzed and finally returns the whole parsed data stored in soup. A screenshot/preview of the output is given below.

```
<html class="responsive DesktopUI" lang="en"><head>
<meta content="text/html; charset=utf-8" http-equiv="Content-Type"/>
<meta content="width=device-width,initial-scale=1" name="viewport"/>
<meta content="#171a21" name="theme-color"/>
<title>Steam Search</title>
<link href="/favicon.ico" rel="shortcut icon" type="image/x-icon"/>
<link href="https://store.fastly.steamstatic.com/public/shared/css/motiva_sans.css?v=YzJgj1FjzW34&l=english&_cdn=fastly" rel="stylesheet" type="text/css"/>
<link href="https://store.fastly.steamstatic.com/public/shared/css/shared_global.css?v=F0AmfT9E8goz&l=english&_cdn=fastly" rel="stylesheet" type="text/css"/>
<link href="https://store.fastly.steamstatic.com/public/shared/css/buttons.css?v=BZhNETESfYSJ&l=english&_cdn=fastly" rel="stylesheet" type="text/css"/>
<link href="https://store.fastly.steamstatic.com/public/css/v6/store.css?v=QQDgnRsK7TU9&l=english&_cdn=fastly" rel="stylesheet" type="text/css"/>
<link href="https://store.fastly.steamstatic.com/public/css/v6/browse.css?v=ZWD20rh07eSv&l=english&_cdn=fastly" rel="stylesheet" type="text/css"/>
<link href="https://store.fastly.steamstatic.com/public/css/v6/search.css?v=TqOGgK4VYgHm&l=english&_cdn=fastly" rel="stylesheet" type="text/css"/>
<link href="https://store.fastly.steamstatic.com/public/shared/css/shared_responsive.css?v=FZmjIuAkX6zJ&l=english&_cdn=fastly" rel="stylesheet" type="text/css"/>
<script src="https://store.fastly.steamstatic.com/public/shared/javascript/jquery-1.8.3.min.js?v=NXam5zydzNu8&l=english&_cdn=fastly" type="text/javascript"></script>
<script type="text/javascript">$J = jQuery.noConflict();</script><script type="text/javascript">VALVE_PUBLIC_PATH = "https://store.fastly.steamstatic.com/public/";</script><script src="https://store.fastly.steamstatic.com/public/shared/javascript/tooltip.js?v=k5le07MVvAaV&l=english&_cdn=fastly" type="text/javascript"></script>
<script src="https://store.fastly.steamstatic.com/public/shared/javascript/shared_global.js?v=AXJWzWE--2g&l=english&_cdn=fastly" type="text/javascript"></script>
```

```
games = soup.find_all("a", class_="search_result_row")
print("\nGames found after dynamic scrolling:", len(games))
```



STEAM

Task 1 : Web Scraping Steam Store

The code in the previous page finds all game entries from the Steam webpage by searching for <a> tags with the class "search_result_row" and prints the total number of games found after scrolling loaded additional content dynamically (19 times). The output is given below to the right.

Games found after dynamic scrolling: 1000

```
titles = []
release_dates = []
prices = []
reviews = []
discounts = []
links = []
genre_list = []
for game in games:
    # Titles.
    title = game.find("span", class_="title")
    if title:
        title = title.text.strip()
    else:
        title = "N/A"
    # Release Dates.
    release = game.find("div", class_="search_released")
    if release:
        release = release.text.strip()
    else:
        release = "N/A"
    # Prices.
    price = game.find("div", class_="discount_final_price")
    if price:
```

```
price = price.text.strip()
    else:
        price = "Free"
    titles.append(title)
    release_dates.append(release)
    prices.append(price)
    # Reviews.
    review = game.find("span", class_="search_review_summary")
    if review:
        review = review.get("data-tooltip-html")
        if review:
            review = review.replace("<br>", " : ")
        else:
            review = "N/A"
    else:
        review = "N/A"
    reviews.append(review)
    # Discounts.
    discount = game.find("div", class_="discount_pct")
    if discount:
        discount = discount.text.strip()
    else:
        discount = "0%"
```



Task 1 : Web Scraping Steam Store

```
# Links.
    link = game.get("href")
    if link:
        link = link.strip()
    else:
        link = "N/A"
    links.append(link)
# Using Steam Official API to retrieve game genres and Nested data extraction.
    # 1. Extracting the current app_id.
    app_id = game.get("data-ds-appid")
    if app_id:
        try:
            # 2. Querying the Steam API.
            url = f"https://store.steampowered.com/api/appdetails?appids={app_id}"
            response = requests.get(url)
            data = response.json()
            # 3. Extracting nested data safely.
            game_data = data.get(str(app_id))
            if game_data and game_data.get("success"):
                inner_data = game_data.get("data", {})
                genres = inner_data.get("genres", [])
                if genres:
                    genre_names = [genre["description"] for genre in genres if "description" in genre]
                    genre_list.append(", ".join(genre_names))
                else:
                    genre_list.append("Unknown")
            else:
                genre_list.append("Unknown")
        except Exception as e:
            print(f"Error fetching AppID {app_id}: {e}")
```




Task 1 : Web Scraping Steam Store

```
genre_list.append("N/A")
    # This block runs after the try/except finishes, for every different App Id.
    print(f"Done: {app_id}")
    time.sleep(0.2)
else:
    genre_list.append("N/A")
```

^ The code above is the continuation from the last two pages and it does the following :

1. Creates empty lists to store game information like titles, release dates, prices, reviews, discounts, links, and genres.
 2. Loops through each game found on the Steam page.
 3. Extracts the game title, release date, and price from the webpage.
 4. Extracts review details and formats them properly.
 5. Extracts discount percentages for games on sale.
 6. Extracts the direct Steam store link for each game.
 7. Retrieves the game's app_id from Steam.
 8. Uses the Steam Official API to fetch extra details about the game.
 9. Extracts genre information from nested JSON API data safely.
 10. Stores all extracted data into their respective lists.
 11. Handles missing data and errors using default values and try-except.
 12. Prints the completed App ID and waits briefly to avoid sending too many API requests too quickly.
- A preview of the output is given in the next page to the left.



Task 1 : Web Scraping Steam Store

Done: 2483190
Done: 730
Done: 1962700
Done: 2215200
Done: 1172470
Done: 578080
Done: 3321460
Done: 264710
Done: 2767030
Done: 230410
Done: 1174180
Done: 3041230
Done: 2183900
Done: 2344520
Done: 3892270
Done: 570
Done: 2605790
Done: 3240220

```
df = pd.DataFrame({ "Title" : titles, "Release_Date" : release_dates, "Price" : prices,
"Reviews" : reviews,"Discount" : discounts, "Game_Link" : links, "Genres" : genre_list })
df.head()
```

^ The code above creates a Pandas DataFrame using the extracted game data such as titles, release dates, prices, reviews, discounts, links, and genres, then displays the first 5 rows of the dataset using `df.head()`. The output is given below.

	Title	Release_Date	Price	Reviews	Discount	Game_Link	Genres
0	Forza Horizon 6	18 May, 2026	₹5,499.00	Very Positive : 86% of the 19,559 user reviews...	0%	https://store.steampowered.com/app/2483190/For...	Racing, Simulation, Sports
1	Counter-Strike 2	21 Aug, 2012	Free	Very Positive : 86% of the 2,542,008 user revi...	0%	https://store.steampowered.com/app/730/Counter...	Action, Free To Play
2	Subnautica 2	14 May, 2026	₹1,800.00	Very Positive : 93% of the 52,404 user reviews...	0%	https://store.steampowered.com/app/1962700/Sub...	Action, Adventure, Early Access
3	LEGO® Batman™: Legacy of the Dark Knight	22 May, 2026	₹3,899.00	Overwhelmingly Positive : 96% of the 2,611 use...	0%	https://store.steampowered.com/app/2215200/LEG...	Action, Adventure
4	Apex Legends™	4 Nov, 2020	Free	Mostly Positive : 76% of the 442,632 user revi...	0%	https://store.steampowered.com/app/1172470/Ape...	Action, Adventure, Free To Play

```
df.to_csv("Steam_Games.csv", index=False)
```

^ The code above Saves the Pandas DataFrame into a CSV file named "Steam_Games.csv" without including row index numbers.



Task 1 : Web Scrapping Steam Store

```
driver.quit()
```

^ The code above closes the Selenium browser session and safely shuts down the Chrome driver to free system resources.



Task 2 : Cleaning & Preprocessing

```
import pandas as pd
import numpy as np
gd = pd.read_csv("Steam_Games.csv")
```

< The code on the left imports the pandas and numpy libraries, then reads the "Steam_Games.csv" file into a DataFrame named gd for data analysis and processing.

```
gd.head()
```

< The code on the left imports the pandas and numpy libraries, then reads the "Steam_Games.csv" file into a DataFrame named gd for data analysis and processing. The output is same as df.head().

```
gd.info()
```

< The code on the left displays information about the DataFrame, including column names, number of rows, data types, and count of non-null (non-missing) values. The output is given in the next page on the left.



Task 2 : Cleaning & Preprocessing

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Title       100 non-null    object
1   Release_Date 100 non-null    object
2   Price       100 non-null    object
3   Reviews     99 non-null     object
4   Discount    100 non-null    object
5   Game_Link   100 non-null    object
6   Genres      100 non-null    object
dtypes: object(7)
memory usage: 5.6+ KB
```

```
# Checking the dataset for missing values.
gd.isnull().sum()
```

< The code right beside checks each column in the dataset for missing (null) values and shows the total number of missing values in every column. The output is given right beside.

```
Title       0
Release_Date
0
Price       0
Reviews     9
Discount    0
Game_Link   0
Genres     127
dtype: int64
```

```
# Replacing the duplicates values in Genres column with "Unknown".
gd["Genres"] = gd["Genres"].fillna("Unknown")
```

< The code on the right replaces missing (NaN) values in the "Genres" column with "Unknown" to handle incomplete data.

```
# Checking for duplicate entries.
gd.duplicated().sum()
```

< The code on the right checks the dataset for duplicate rows and prints the total number of duplicate entries found. The output is in the next page on the top.



Task 2 : Cleaning & Preprocessing

`np.int64(0)`

```
# Cleaning the Price column and converting it into numeric format.
gd["Price"] = (gd["Price"].astype(str).str.replace("₹", "", regex=False).str.replace(",","",regex=False)
               .str.replace("Free", "0", regex=False).str.strip())
gd["Price"] = pd.to_numeric(gd["Price"], errors="coerce")
# Cleaning the Discount column and converting it into numeric format.
gd["Discount"] = (gd["Discount"].astype(str).str.replace("%", "", regex=False).str.replace("-", "", regex=False))
gd["Discount"] = pd.to_numeric(gd["Discount"],errors="coerce")
# Converting Release Date into proper datetime format.
gd['Release_Date'] = gd['Release_Date'].str.strip()gd['Release_Date'] = pd.to_datetime(gd['Release_Date'],format='%d
%b, %Y', errors='coerce')
# Cleaning the Reveiws column.
gd["Reviews"] = (gd["Reviews"].str.split(":").str[0].str.strip())
gd["Reviews"] = (gd["Reviews"].astype(str).str.strip())
print(gd["Price"])
print(gd["Discount"])
print(gd["Release_Date"])
print(gd["Reviews"])
```



Task 2 : Cleaning & Preprocessing

The code in the last page works on the following :

1. Cleans the "Price" column by removing symbols like ₹, commas, and replacing "Free" with 0, then converts it into numeric format.
2. Cleans the "Discount" column by removing % and - symbols, then converts it into numeric format.
3. Cleans and converts the "Release_Date" column into proper datetime format for date analysis.
4. Cleans the "Reviews" column by extracting only the review category text and removing extra spaces.
5. Prints the cleaned "Price", "Discount", "Release_Date", and "Reviews" columns to verify the processed data.

A preview of the output is given right beside.

```
0      5499.0
1         0.0
2      1800.0
3      3899.0
4         0.0
...
95     5000.0
96         0.0
97        95.0
98         0.0
99      400.0
Name: Price, Length: 100, dtype: float64
0         0
1         0
2         0
3         0
4         0
..
95         0
96         0
97        80
98         0
99         0
Name: Discount, Length: 100, dtype: int64
```



Task 2 : Cleaning & Preprocessing

```
# Creating Review score.
review_map = {"Overwhelmingly Positive": 100, "Very Positive": 85, "Positive": 75, "Mostly
Positive": 65, "Mixed": 50, "Mostly Negative": 35, "Negative": 20, "Very Negative": 10}
gd["Review_Score"] = (gd["Reviews"].map(review_map))
# Extracting Year and Month from Release Date.
gd["Release_Year"] = (gd["Release_Date"].dt.year)
gd["Release_Month"] = (gd["Release_Date"].dt.month_name())
# Creating a Price category as Free or Paid and a Discount category.
gd["Game_Type"] = np.where(gd["Price"] == 0, "Free", "Paid")
gd["Discount_Level"] = pd.cut(gd["Discount"], bins=[-1, 0, 25, 50, 75, 100],
labels=["NoDiscount", "Low", "Medium", "High", "Huge"])
# Counting the total no of Genres.
gd["Genre_Count"] = (gd["Genres"].astype(str).apply(lambda x: len(x.split(","))))
```

^ The code above creates a Review_Score column by converting review categories (like Very Positive or Mixed) into numeric scores, extracts the release year and release month from the Release_Date column. It also creates a Game_Type column to classify games as Free or Paid based on price, a Discount_Level column by grouping discounts into categories like Low, Medium, High, and Huge and a Genre_Count column to count how many genres each game has.

```
gd.head()
gd.info()
gd.describe()
```

< The code on the right uses gd.head() and gd.info() to get a look at the data after being cleaned and modified and to double check. gd.describe() → Generates statistical summaries for numeric columns, including count, mean, standard deviation, minimum, maximum, and quartile values.



Task 3 : EDA & Visualizations

```
import matplotlib.pyplot as plt
import seaborn as sb
gd[["Price", "Discount", "Review_Score"]].describe()
```

^ The code above imports matplotlib.pyplot for creating charts and graphs, and imports seaborn for advanced statistical data visualizations with improved styling and Generates statistical summaries specifically for the Price, Discount, and Review_Score columns, including count, mean, standard deviation, minimum, maximum, and quartile values. The output is given below to the left.

	Price	Discount	Review_Score
count	100.000000	100.000000	99.000000
mean	1347.260000	12.520000	78.333333
std	1362.531405	26.186001	16.629210
min	0.000000	0.000000	35.000000
25%	190.250000	0.000000	65.000000
50%	922.500000	0.000000	85.000000
75%	2286.500000	0.000000	85.000000
max	5499.000000	86.000000	100.000000

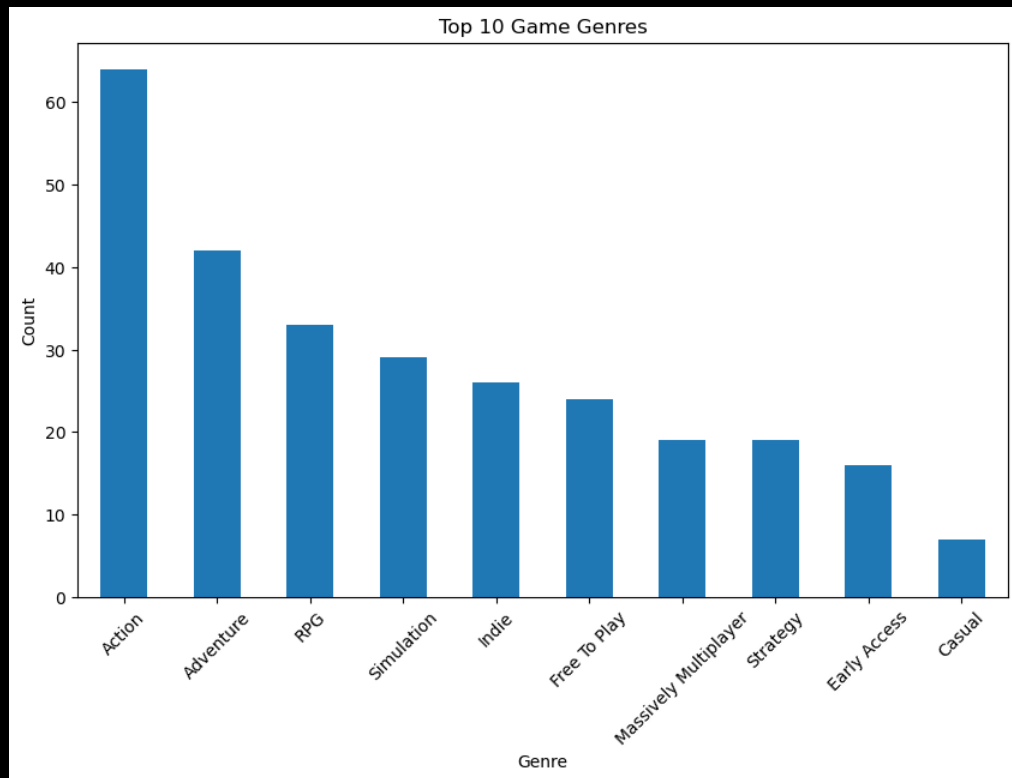
```
# Bar chart to get the top 10 Genres.
genre_counts = (
    gd["Genres"].str.split(", ").explode()
    .value_counts().head(10))
plt.figure(figsize=(10,6))
genre_counts.plot(kind="bar")
plt.title("Top 10 Game Genres")
plt.xlabel("Genre")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```

< The code on the right splits multiple genres for each game into separate entries, counts how many times each genre appears, and selects the top 10 most common genres also creates a bar chart showing the top 10 game genres with genre names on the x-axis and their counts on the y-axis and adjusts the figure size, chart title, axis labels, and rotates genre labels for better readability, then displays the chart. The Bar chart is given in the next page on the top left.



STEAM

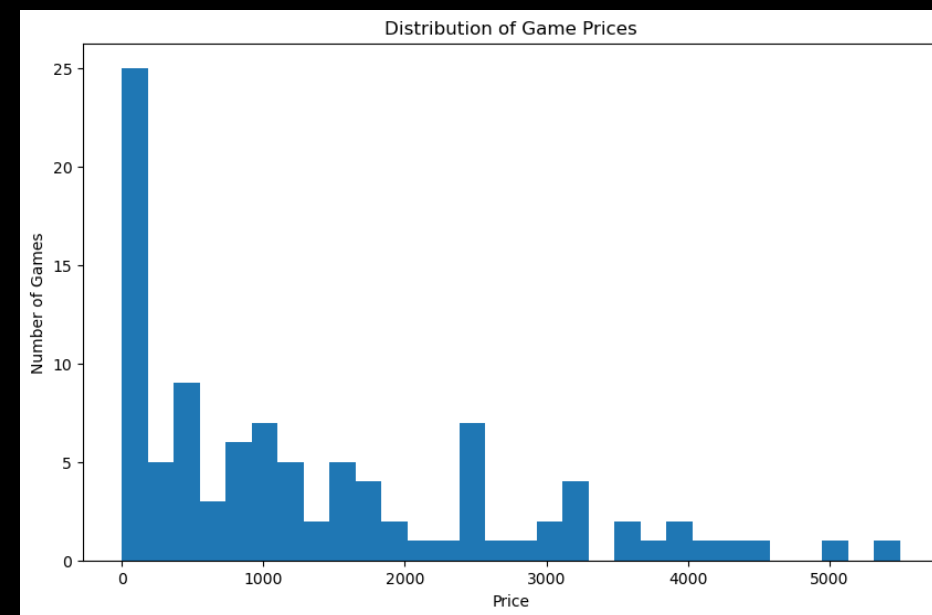
Task 3 : EDA & Visualizations



```
# Histogram to understand the Price distribution.
plt.figure(figsize=(10,6))plt.hist(gd["Price"].dropna(),bins=30)
plt.title("Distribution of Game Prices")
plt.xlabel("Price")
plt.ylabel("Number of Games")
plt.show()
```

^ The code above creates a histogram to visualize the distribution of game prices in the dataset, removes missing price values, divides prices into 30 bins, and shows how many games fall within each price range and adds a chart title and axis labels, then displays the histogram. The Histogram is given below.

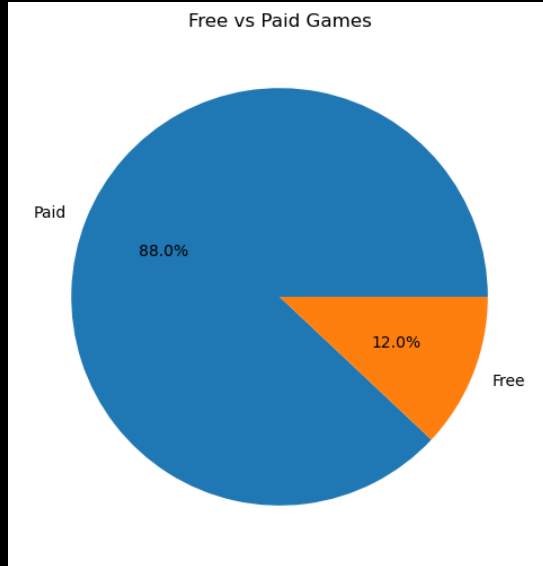
Insights from the Histogram : The distribution is right-skewed, meaning most games are priced in the lower price range, while only a few games have very high prices, a large concentration of games is seen between approximately ₹0–₹1000, the number of games decreases as the price increases and there are a few high-priced outliers around ₹4000–₹5500. This suggests that the platform mainly contains affordable games, with only a small number of premium-priced titles.





STEAM

Task 3 : EDA & Visualizations



The code on the right counts the number of games released each year and sorts the years in chronological order, creates a line chart to visualize yearly game release trends over time and adds markers for each year, customizes x-axis labels, titles, axis labels, and grid lines, then displays the chart. The Line chart is given in the next page.

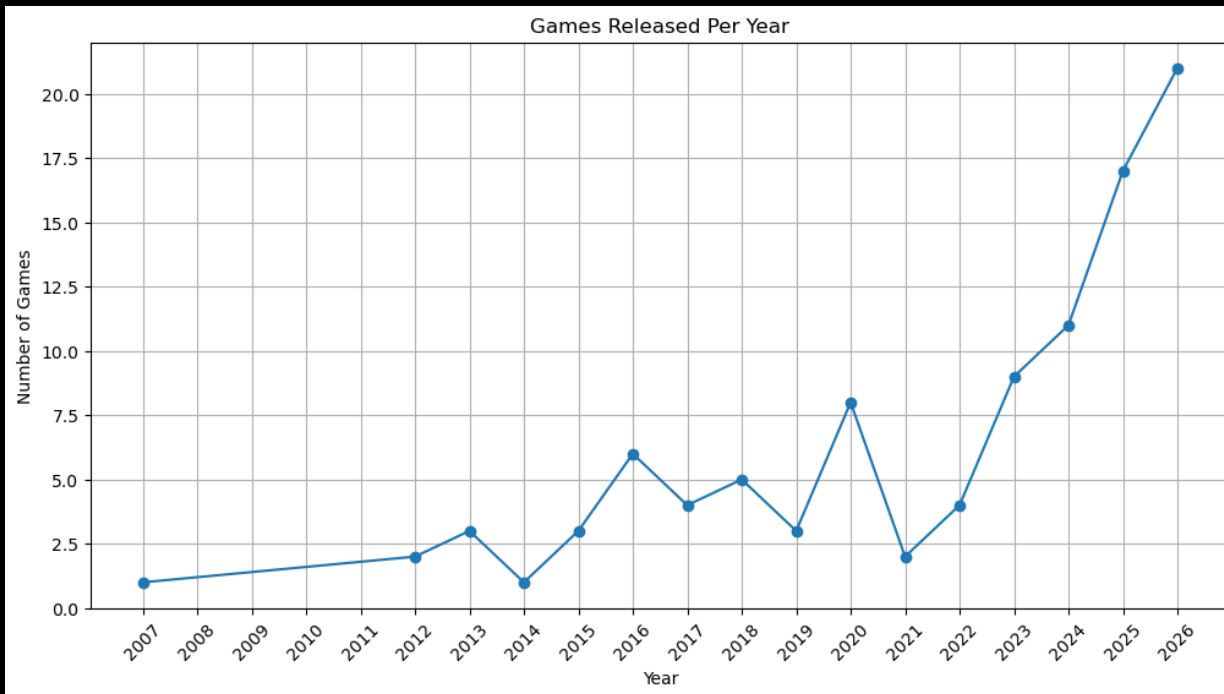
```
# Pie chart for Free vs Paid games.
game_type_counts = (gd["Game_Type"].value_counts())
plt.figure(figsize=(6,6))
plt.pie(game_type_counts,labels=game_type_counts.index,autopct="%1.1f%%")
plt.title("Free vs Paid Games")
plt.show()
```

^ The code above counts the number of "Free" and "Paid" games in the dataset, creates a pie chart to show the proportion of free vs paid games and displays percentage values on the chart, adds labels and a title, then shows the pie chart. The Pie chart is given just on the left.

```
# Line chart for Yearly Release Trend Analysis.
year_counts = (gd["Release_Year"].value_counts().sort_index())
plt.figure(figsize=(14,7))
plt.plot(year_counts.index,year_counts.values, marker="o")
plt.xticks(range(int(gd['Release_Year'].min()),
int(gd['Release_Year'].max()) + 1),
rotation=45)
plt.title("Games Released Per Year")
plt.xlabel("Year")
plt.ylabel("Number of Games")
plt.grid(True)
plt.show()
```



Task 3 : EDA & Visualizations



Insights from the Line chart : The number of game releases shows an overall upward trend over the years, between 2007–2015, relatively few games were released each year, from 2016 onwards, game releases became more frequent with some fluctuations and a significant growth is visible after 2022, indicating rapid expansion in game production or platform popularity. The highest number of releases occurs in 2026, suggesting continued growth in the gaming market.

```
# Boxplot for Non-zero Discount Analysis and Outlier detection.
active_disc = gd[gd["Discount"]>0]
plt.figure(figsize=(10,6))
sb.boxplot(x=active_disc["Discount"])
plt.title("Non-Zero Discount Distribution")
plt.show()
```

< The code on the left filters the dataset to include only games with discounts greater than 0, creates a boxplot to analyze the distribution of active discounts and detect outliers and adds a chart title and displays the boxplot. The Boxplot is given in the next page.



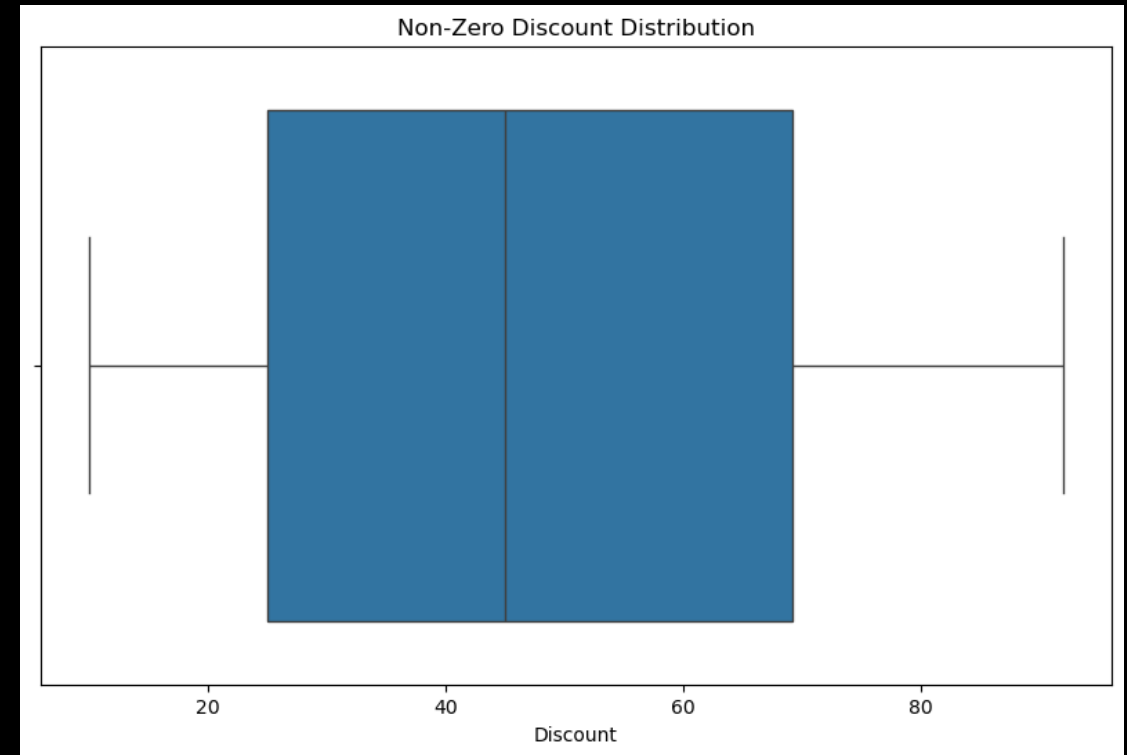
STEAM

Task 3 : EDA & Visualizations

Insights from the Boxplot : The discounts are widely spread, ranging from around 10% to 90%, the median discount is approximately 45–50%, meaning most discounted games offer moderate discounts, the middle 50% of discounts lie roughly between 25% and 70%, showing high variability in discount offers and no major outliers are visible, indicating discounts are relatively evenly distributed without extreme unusual values. This suggests that the platform frequently provides medium to high discounts on games.

```
# Violin plot to understand the Price distribution for each
Review Score.
fig, ax = plt.subplots(figsize=(10, 6))
sb.set_theme(style="whitegrid")
ax = sb.violinplot(data=gd,x="Review_Score",y="Price",

inner="box",cut=0,palette="Blues",hue="Review_Score",ax=ax)
sb.move_legend(ax, "upper left", bbox_to_anchor=(1.05, 1),
title="Review Score")
ax.set_title("Price Distribution by Review Score",
fontsize=14, pad=15)
ax.set_xlabel("Review Score", fontsize=12)
ax.set_ylabel("Price", fontsize=12)
plt.tight_layout()
plt.show()
```

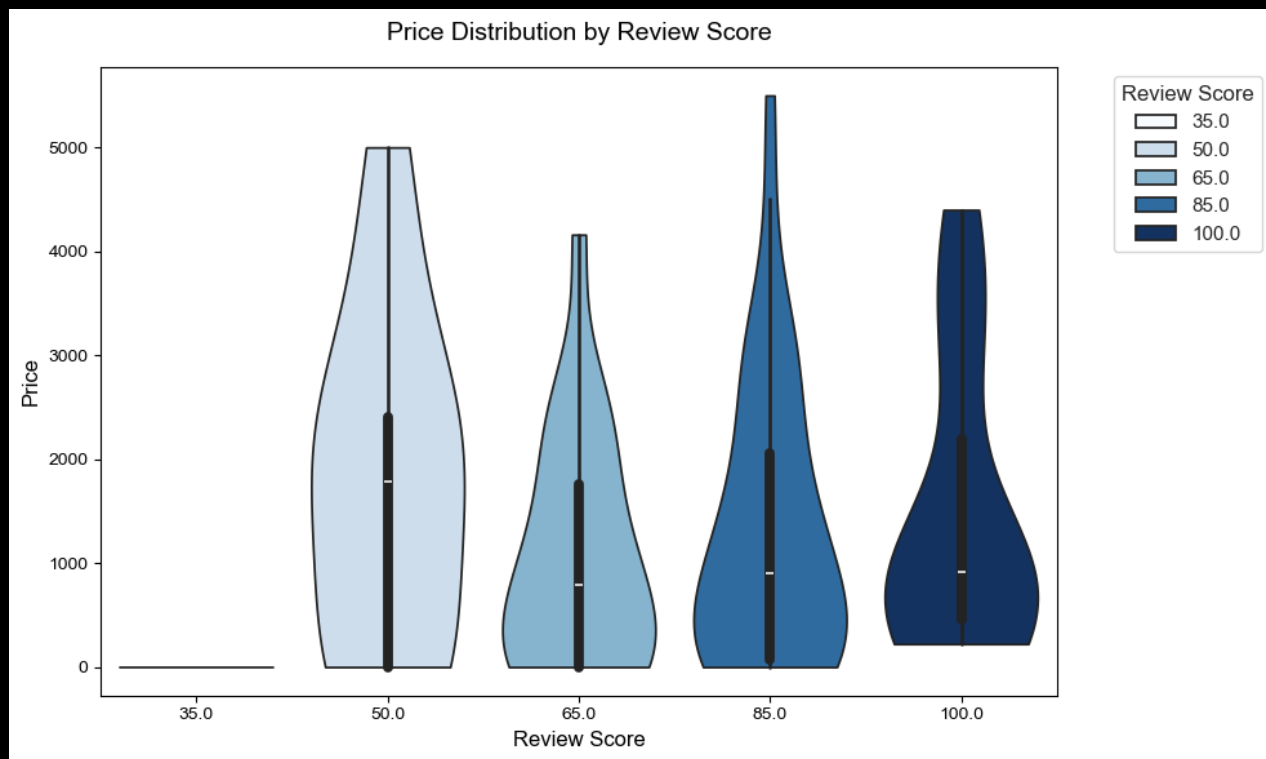


< The code on the left creates a violin plot to visualize how game prices are distributed across different review scores, uses Seaborn styling and adds an inner boxplot inside each violin for better statistical insight, colors the plot based on review scores, adds a legend, chart title, axis labels and adjusts the layout for better spacing and displays the chart. The Violin plot is given in the next page.



STEAM

Task 3 : EDA & Visualizations



Insights from the Violin plot :

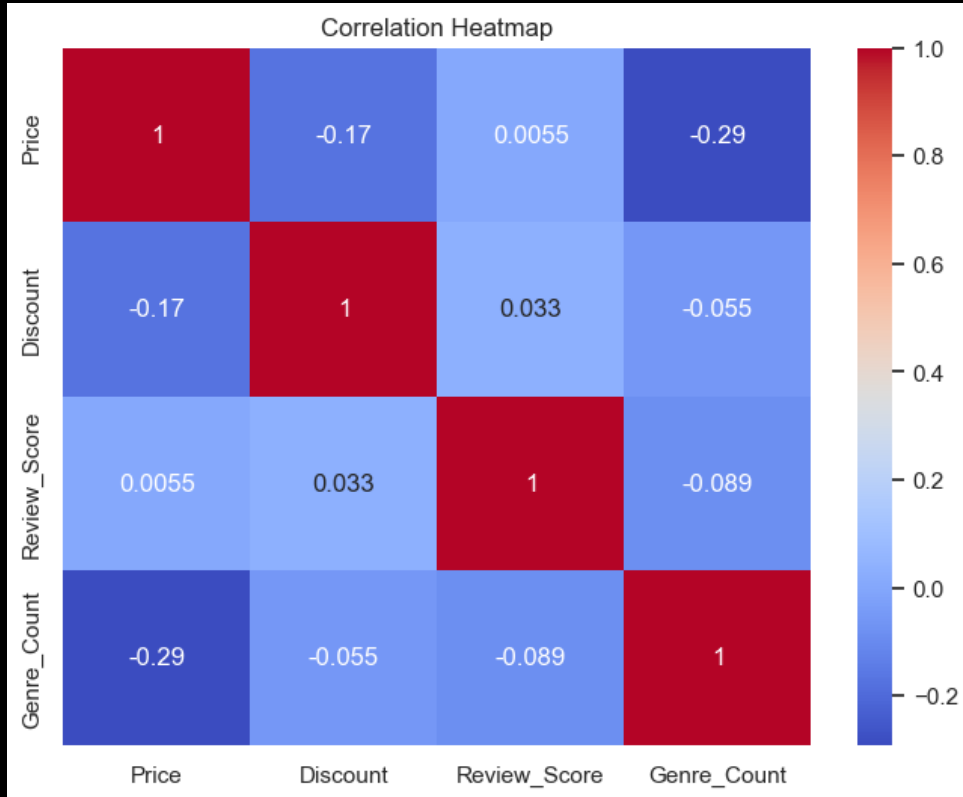
1. Games with higher review scores (85–100) tend to have a wider and generally higher price distribution.
2. Most games across all review categories are concentrated in the lower to mid-price range.
3. Review score 50 shows a large spread in prices, including some very expensive games.
4. The 35 review score category has very few games and almost no price variation.
5. Overall, highly rated games appear to include more premium-priced titles, but price alone does not guarantee better reviews.

```
# Generating Heatmap to get the Correlation Matrix between the Numerical columns.  
num_gd = gd[["Price", "Discount", "Review_Score", "Genre_Count"]]  
plt.figure(figsize=(8,6))  
sb.heatmap(num_gd.corr(), annot=True, cmap="coolwarm")  
plt.title("Correlation Heatmap")  
plt.show()
```

< The code on the right selects the numerical columns Price, Discount, Review_Score, and Genre_Count from the dataset, then it calculates the correlation matrix to measure relationships between these numerical features and creates a heatmap to visually display the correlation values using color intensity. It shows correlation values inside the heatmap cells, adds a title, and displays the chart. The Heatmap is in the next page.



Task 3 : EDA & Visualizations



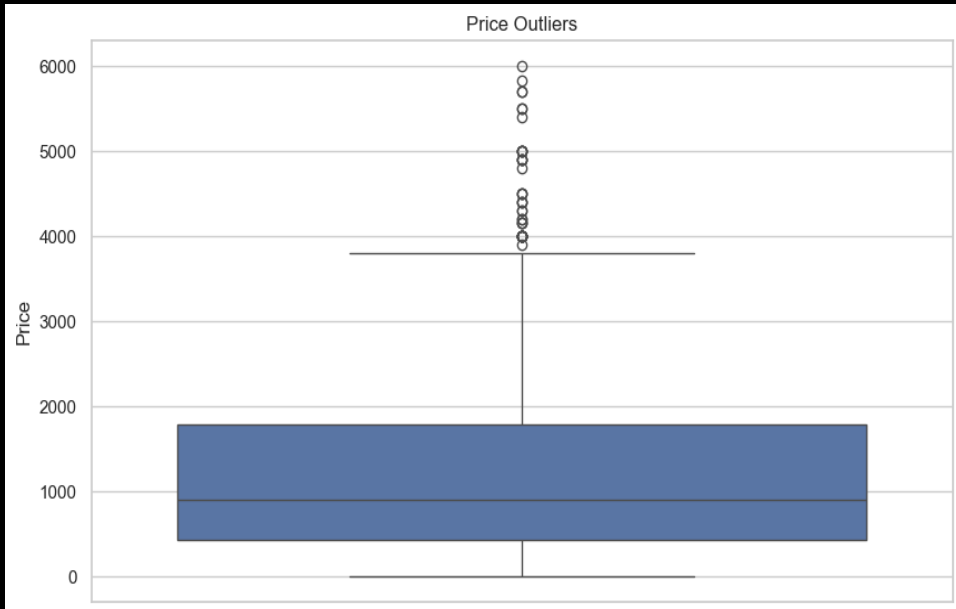
Insights from the Heatmap : Most variables show very weak correlations, meaning they are not strongly related to each other, Price and Genre Count have the strongest relationship with a correlation of around -0.29, indicating games with more genres may tend to have slightly lower prices. Price and Discount show a weak negative correlation (-0.17), suggesting higher-priced games may receive slightly smaller discounts. Review Score has almost no correlation with Price or Discount, meaning ratings are largely independent of pricing and discounts. Overall, no strong positive or negative relationships are present in the dataset.

```
# Boxplot to identify the outliers in Price column.
plt.figure(figsize=(10,6))
sb.boxplot(y=gd["Price"])
plt.title("Price Outliers")
plt.show()
```

< The code on the left creates a boxplot for the "Price" column to visualize the distribution of game prices and identify outliers, displays unusually high or low price values as points outside the boxplot range and adds a chart title and shows the visualization. The Boxplot is given in the next page.



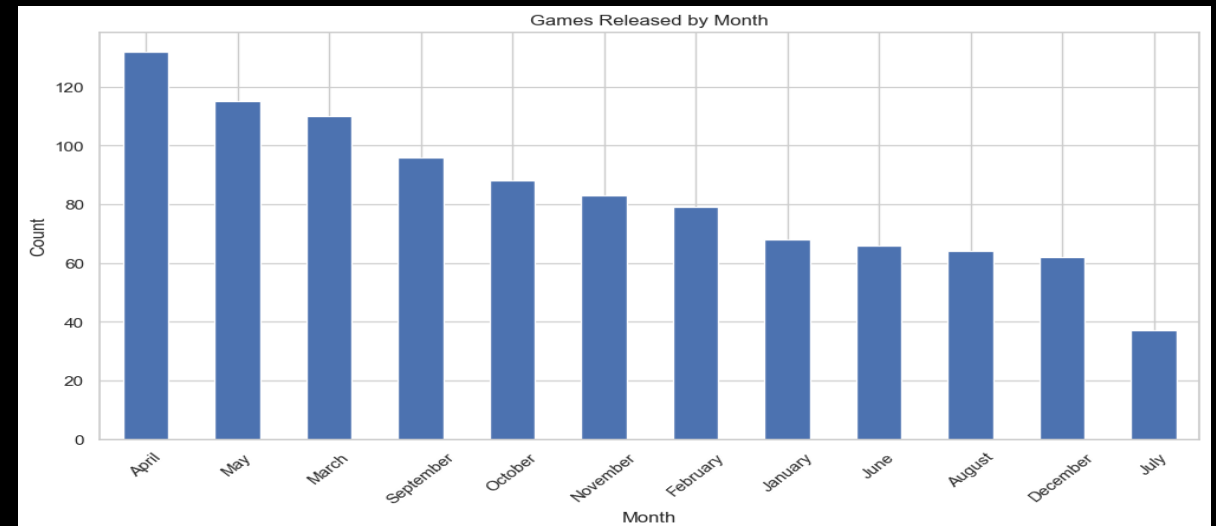
Task 3 : EDA & Visualizations



Insights from the Boxplot : shows that most game prices are concentrated in the lower price range, as indicated by the lower median, there are several high-price outliers above approximately ₹3800, reaching up to around ₹6000. The dataset has a right-skewed distribution, meaning only a small number of games are very expensive, the wide spread of the box indicates significant variation in game prices. Premium-priced games are uncommon compared to moderately priced games.

< The code below to the left Counts the number of games released in each month using the Release_Month column, Creates a bar chart to visualize monthly game release trends and adds chart title, axis labels, and rotates month labels for better readability, then displays the chart. The Bar chart is given below.

```
# Bar plot to get the Monthly Release Trend.
month_counts = (gd["Release_Month"].value_counts())
plt.figure(figsize=(12,6))
month_counts.plot(kind="bar")
plt.title("Games Released by Month")
plt.xlabel("Month")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```

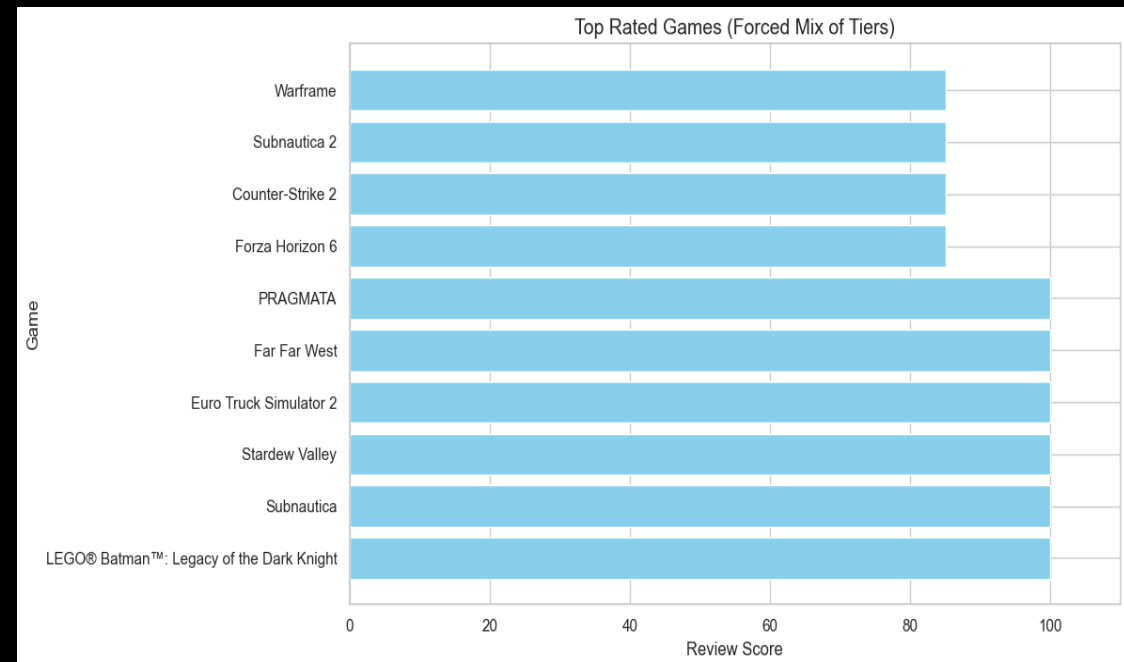




Task 3 : EDA & Visualizations

```
# Identifying the Top Rated/Reviewed by the top 2 tiers of
Review Score using
# a Horizontal Bar Chart.
top_tier = gd[gd["Review_Score"] == 100]
second_tier = gd[gd["Review_Score"] == 85]
sampled_top = top_tier.head(6)
sampled_second = second_tier.head(4)
top_games = pd.concat([sampled_top, sampled_second])
top_games = top_games.sort_values(by="Review_Score",
ascending=False)
plt.figure(figsize=(12, 6))
plt.barh(top_games["Title"], top_games["Review_Score"],
color="skyblue")
plt.title("Top Rated Games (Forced Mix of Tiers)",
fontsize=14)
plt.xlabel("Review Score", fontsize=12)
plt.ylabel("Game", fontsize=12)
plt.xlim(0, 110)
plt.tight_layout()
plt.show()
```

< The code on the left filters games with the top two review score tiers (100 and 85), selects sample games from both tiers and combines them into one dataset, sorts the games by review score in descending order, creates a horizontal bar chart to compare the review scores of the selected top-rated games and adds chart title, axis labels, adjusts the x-axis range, and displays the chart. The bar chart is given below.



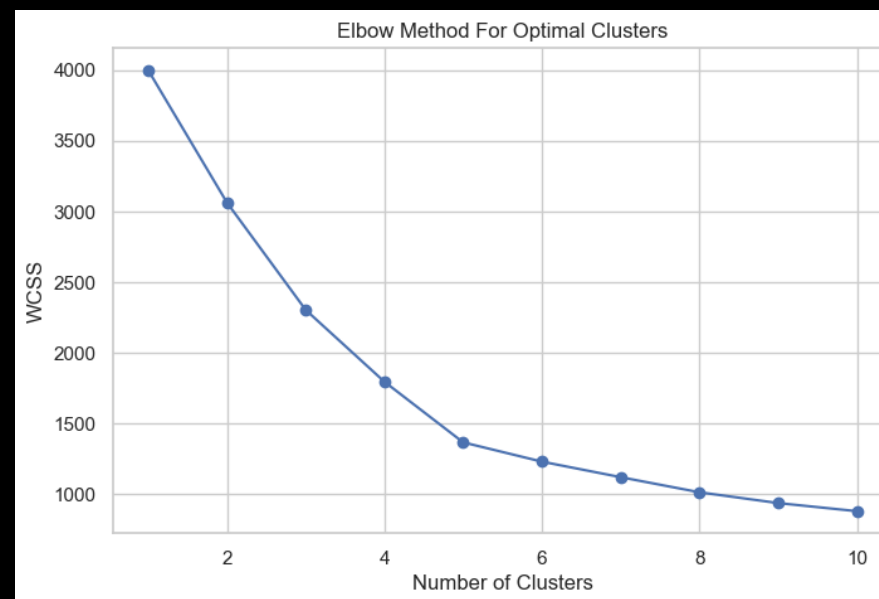


STEAM

Task 4 : Model Building & Evaluation

```
import warnings
warnings.filterwarnings("ignore")
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
# Selecting the features.
features = ["Price", "Discount",
            "Genre_Count", "Release_Year"]
X = gd[features]
# Handling missing values.
X = X.fillna(X.mean())
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Using the Elbow Method to find the optimal value of K.
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42,
                    n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
# Generating the Elbow Graph.
plt.figure(figsize=(8,5))
plt.plot(range(1,11), wcss, marker="o")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method For Optimal Clusters")
plt.grid(True)
plt.show()
```

< The code on the left imports libraries for warnings handling, clustering (KMeans), data scaling (StandardScaler), and dimensionality reduction (PCA), selects numerical features like Price, Discount, Genre_Count, and Release_Year for clustering analysis. Handles missing values by replacing them with the column mean, standardizes the feature values so all features are on the same scale, uses the K-Means clustering algorithm with different cluster counts (1–10). It also calculates WCSS (Within-Cluster Sum of Squares) for each cluster count to measure clustering performance and creates an Elbow Method graph to help identify the optimal number of clusters for the dataset.





STEAM

Task 4 : Model Building & Evaluation

```
# Developing the K-Means Model.
kmeans = KMeans(n_clusters=4,
                random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)
# Adding Clusters to the DataFrame.
gd["Cluster"] = clusters
# Implementing PCA 2D Visuals.
pca = PCA(n_components=2)
pca_features = pca.fit_transform(X_scaled)
# Interpreting and adding the accurate label to each cluster.
cluster_names = {0: "Old Discounted Games", 1: "Budget Indie Games", 2: "Diverse Modern Games", 3: "Premium AAA Games"}
# Mapping the labels.
gd["Cluster_Name"] = gd["Cluster"].map(cluster_names)
print(gd[["Title", "Cluster", "Cluster_Name"]].head(10))
# Visualizing the Clusters.
# Create figure
plt.figure(figsize=(10,6))
# Plotting clusters.
for cluster in sorted(gd["Cluster"].unique()):
    plt.scatter(pca_features[gd["Cluster"] == cluster, 0],
                pca_features[gd["Cluster"] == cluster, 1],
                label=f"{cluster} - {cluster_names[cluster]}",
                s=60, alpha=0.8)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title("Steam Game Clusters")
plt.legend(title="Clusters")
plt.grid(True)
plt.tight_layout()
```



Task 4 : Model Building & Evaluation

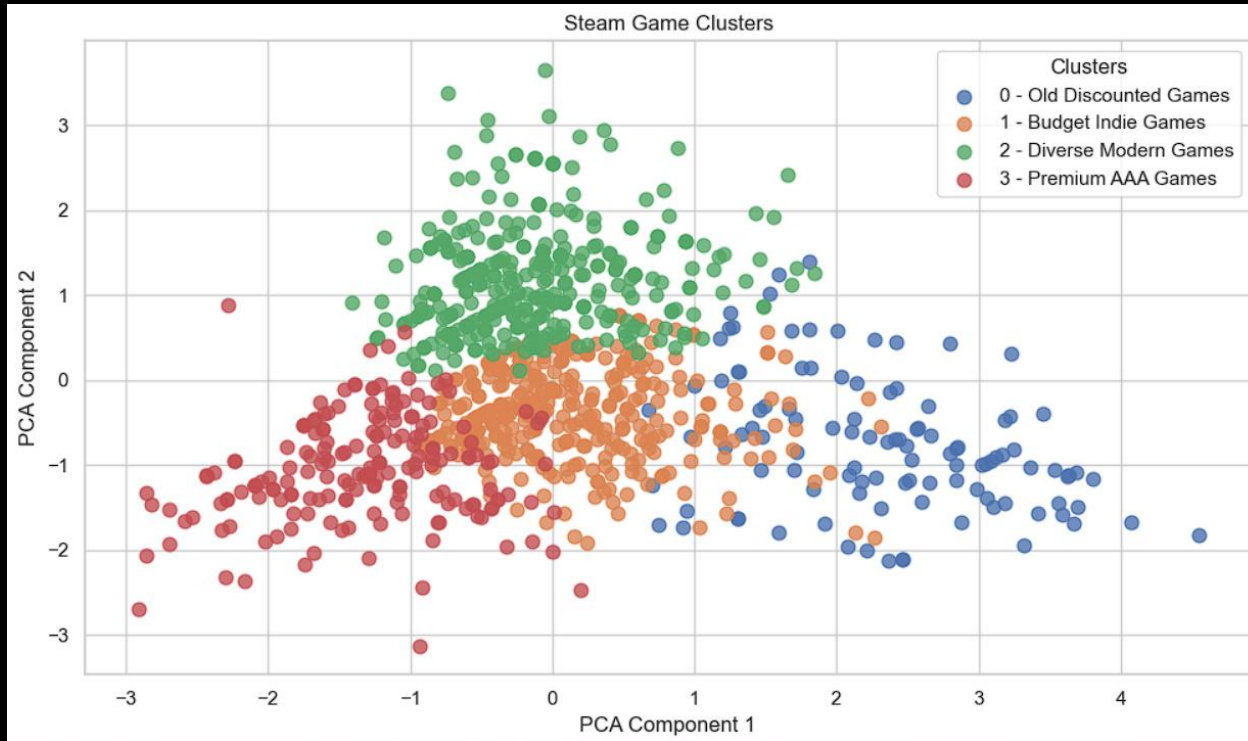
```
plt.show()
# Displaying Cluster Summary.
print("\nCluster Counts:\n")
print(gd["Cluster"].value_counts())
# Average values per Cluster.
cluster_summary = gd.groupby("Cluster")[features].mean()
print("\nCluster Summary:\n")
print(cluster_summary)
```

	Title	Cluster	Cluster_Name
0	Forza Horizon 6	3	Premium AAA Games
1	Counter-Strike 2	1	Budget Indie Games
2	Subnautica 2	2	Diverse Modern Games
3	LEGO® Batman™: Legacy of the Dark Knight	3	Premium AAA Games
4	Warframe	1	Budget Indie Games
5	PUBG: BATTLEGROUNDS	2	Diverse Modern Games
6	Apex Legends™	1	Budget Indie Games
7	Subnautica	0	Old Discounted Games
8	Red Dead Redemption 2	0	Old Discounted Games
9	Crimson Desert	3	Premium AAA Games

< The code on the left is a continuation from the previous page, it Creates a K-Means clustering model with 4 clusters and trains it using the scaled feature data, predicts cluster groups for each game and stores them in a new Cluster column. Uses PCA (Principal Component Analysis) to reduce the dataset into 2 dimensions for easier visualization, assigns meaningful names to each cluster, such as "Budget Indie Games" and "Premium AAA Games". Maps the cluster labels into a new Cluster_Name column, displays sample game titles along with their assigned cluster and cluster name, creates a scatter plot to visualize the clusters in 2D PCA space with different labels for each cluster, adds titles, axis labels, legend, grid, and formatting to improve the visualization. Prints the number of games in each cluster, calculates and prints the average feature values (Price, Discount, Genre_Count, Release_Year) for every cluster to summarize cluster characteristics. The output is given on the left will continue till the next next page



Task 4 : Model Building & Evaluation



Cluster Counts:

Cluster

```
1    356
2    350
3    187
0    107
```

Name: count, dtype: int64

Cluster Summary:

	Price	Discount	Genre_Count	Release_Year
Cluster				
0	614.205607	66.943925	2.037383	2019.289720
1	895.994382	2.497191	1.640449	2021.199438
2	773.322857	3.728571	4.222857	2023.262857
3	3265.914439	2.101604	2.080214	2023.251337

Insights from the Clustering Analysis : Cluster 0 – Old Discounted Games – High average discount (~67%), Older release year (~2019) , Moderate pricing, likely older games heavily discounted during sales, Cluster 1 – Budget Indie Games – Lowest prices (~₹896), Very low discounts, Fewer genres per game, mostly affordable indie/smaller-scale games, Cluster 2 – Diverse Modern Games – Highest genre diversity (~4 genres/game), Newer release years (~2023), Medium pricing, represents modern games with mixed gameplay styles and Cluster 3 – Premium AAA Games – Highest average price (~₹3266), Very low discounts, Recent releases and likely premium AAA/high-budget titles.



STEAM

Task 4 : Model Building & Evaluation

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Creating Target variable based on Review_Score.
gd["Popular"] = np.where(gd["Review_Score"] >= 80, 1, 0)
# Setting features and target variable.
features = ["Price", "Discount", "Genre_Count", "Release_Year"]
X = gd[features].copy()
y = gd["Popular"]
# Handling missing values.
for col in X.columns:
    X[col] = X[col].fillna(X[col].median())
# Splitting the Training and Testing data.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)
# Defining the Parameter Grid to search through.
param_grid = {'n_estimators': [50, 100, 200],
              'max_depth': [3, 5, 7, None],
              'min_samples_split': [2, 5, 10]}
# Initializing Grid Search with Cross-Validation, it splits the training data 5 ways
# to validate choices.
rf_base = RandomForestClassifier(random_state=42, class_weight="balanced")
grid_search = GridSearchCV(estimator=rf_base, param_grid=param_grid, cv=5, scoring='accuracy', n_jobs=-1)
# Run the search on training data.
grid_search.fit(X_train, y_train)
# Print out the absolute best combination discovered.
```




STEAM

Task 4 : Model Building & Evaluation

```
print("Best Parameters Found:", grid_search.best_params_)
# Using the best discovered model to make predictions.
best_classifier = grid_search.best_estimator_
y_pred = best_classifier.predict(X_test)
# Model Evaluation.
accuracy = accuracy_score(y_test, y_pred)
print("\nOptimized Accuracy:", accuracy)
print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))
# Visualizing the Confusion Matrix.
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,4))
sb.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Optimized Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

^ The code above is the continuation of the code from the previous page, it imports libraries for data splitting, hyperparameter tuning (GridSearchCV), Random Forest classification, and model evaluation metrics, creates a target column "Popular" where games with Review_Score >= 80 are marked as 1 (popular), otherwise 0. Selects features like "Price", "Discount", "Genre_Count", and "Release_Year" for prediction, handles missing values by replacing them with the median of each column, splits the dataset into training and testing sets while preserving class balance using stratify. Defines different Random Forest parameter combinations to test (n_estimators, max_depth, min_samples_split), uses GridSearchCV with 5-fold cross-validation to find the best model parameters automatically, trains the optimized Random Forest model on the training data, prints the best parameter



Task 4 : Model Building & Evaluation

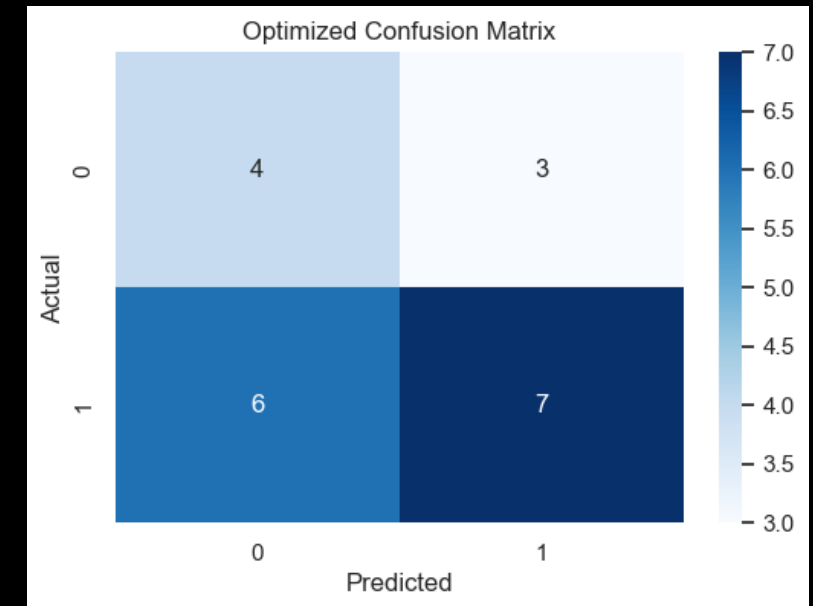
combination found during tuning, predicts whether games are popular or not using the optimized model. Then it evaluates model performance using accuracy score and classification report and creates a confusion matrix heatmap to visualize correct and incorrect predictions made by the model. The output is given below.

```
Best Parameters Found: {'max_depth': 3, 'min_samples_split': 10, 'n_estimators': 50}
```

Optimized Accuracy: 0.55

Classification Report:

	precision	recall	f1-score	support
0	0.40	0.57	0.47	7
1	0.70	0.54	0.61	13
accuracy			0.55	20
macro avg	0.55	0.55	0.54	20
weighted avg	0.59	0.55	0.56	20



Insights from RandomTreeClassifier : The model achieved an accuracy of 55%, which indicates moderate prediction performance, best parameters found: max_depth = 3, min_samples_split = 10, n_estimators = 50. The model performs better at predicting popular games (Class 1) than non-popular games, Precision for popular games is 0.70, meaning many predicted popular games were correct, Recall for popular games is 0.54, so some popular games were still missed, Non-popular games (Class 0) had weaker prediction performance. From the confusion matrix: 7 popular games were correctly identified, 4 non-popular games were correctly identified. The model made several false predictions in both categories. Overall, game popularity cannot be predicted strongly using only: Price, Discount, Genre count and Release year.



Task 5 : Interactive Dashboard

1. Overview/Executive Dashboard :





Task 4 : Model Building & Evaluation

2. Gaming Details/Drill-through Page :



Task 4 : Model Building & Evaluation

3. Final Insights and Cluster Chart:



SteamVision

Insights & Findings

- **Modern Games Dominate the Market.**

Most Steam games in the dataset were released after 2020, indicating rapid industry growth and increased developer activity in recent years.

- **AAA Titles Form a Premium Segment.**

Cluster analysis revealed that AAA games maintain significantly higher average prices, often exceeding ₹3000.

- **Older Games Depend on Heavy Discounts.**

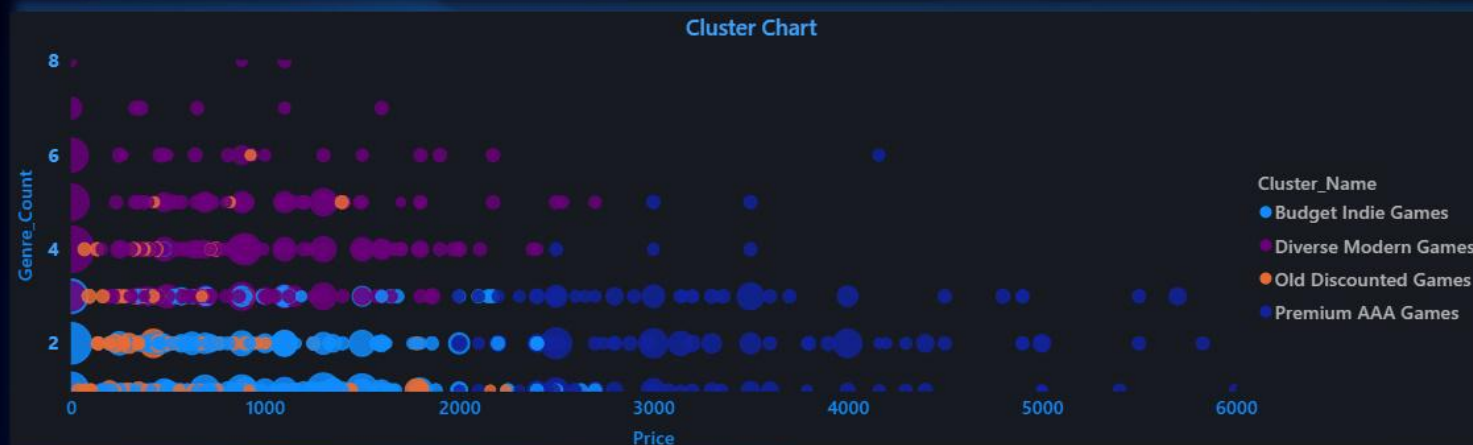
Games released before 2020 tend to offer larger discounts, suggesting discounting strategies help sustain engagement.

- **Genre Diversity Influences Pricing.**

Games with multiple genres generally show higher prices and broader market appeal.

- **Machine Learning Insights.**

Clustering successfully segmented games into meaningful categories such as Budget Indie, Genre-Rich, Discounted Classics, and Premium AAA titles and predicted how different features are affecting the popularity of a game.





Task 5 : Final Report & Video Explanation

Final Recommendations :

1. Improve game recommendation algorithms using player behavior, playtime, and genre preferences for more personalized suggestions.
2. Increase visibility for high-quality indie games through curated promotions and smarter discovery systems.
3. Enhance review authenticity filtering to reduce spam, review bombing, and misleading ratings.
4. Introduce better discount timing strategies instead of relying heavily on large discounts for older games.
5. Provide more support for new AAA releases with dynamic pricing, bundles, and loyalty rewards to improve accessibility.
6. Improve metadata quality by encouraging developers to use more accurate tags and genre classifications.
7. Add stronger analytics tools for developers to understand player engagement and sales trends.
8. Improve the popularity prediction system using additional factors like:
 - a)Active player count
 - b)User engagement
 - c)Community activity
 - d)Streaming popularity
 - e)Wishlist trends
9. Promote multi-genre and innovative games more aggressively since modern diverse games show strong clustering patterns.
10. Optimize seasonal sales and marketing campaigns using machine learning insights from user purchasing behavior.



STEAM

Task 6 : Final Report & Video Explanation

Video Drivelink :

<https://drive.google.com/file/d/1xEUjaSg10vkCpT71HM3ZBTfMeXq6d0RK/view?usp=sharing> :